

Doctorly Take-Home — Assessment Playbook

Window: Saturday 18:00-22:00 (4 hours) · .NET Core + EF Core 6+ · commit to a Git repo · return repo link · **delete repo after the review** **They told you the rubric:** "no right or wrong... how you communicate, what choices you make... document reasonable assumptions." So this is a **judgement + communication test** wearing a coding-task costume. Build as if you must explain every choice — because in the 45-minute review, you will.

The golden rules

1. **Leave it working.** A smaller feature set that builds and runs beats a bigger one that doesn't. Do a clean-clone check before you finish.
 2. **Document every assumption.** They said they won't be on standby — so when the spec is unclear, make a reasonable choice and write it in the README. Documented assumptions score points; silent guesses don't.
 3. **Don't over-engineer.** Four hours does not need Clean Architecture, microservices, or CQRS-with-two-contexts unless the spec demands it. Showing restraint *is* the senior signal. Add a value object or aggregate only if the domain earns it — and say why.
 4. **Depth over breadth.** One feature working end-to-end beats five half-built ones.
 5. **Commit in a readable story.** Small, frequent, well-messaged commits show how you work.
-

Time plan (time-boxed)

- **18:00-18:30 — Understand & plan.** Read the spec twice. List requirements and every ambiguity. Decide MVP scope. Sketch the domain model on paper. Create the repo, commit an empty solution skeleton. *Don't build features yet.*
 - **18:30-19:15 — Skeleton.** Project structure, domain entities, DbContext + Fluent config, first migration. Get it building. Commit.
 - **19:15-21:00 — Core feature, vertical slice.** Build one operation fully end-to-end before starting the next. Commit after each working slice.
 - **21:00-21:30 — Tests.** A handful of meaningful tests on the core business rules. Not coverage-chasing.
 - **21:30-21:50 — README.** The scoring centrepiece (template below).
 - **21:50-22:00 — Finish clean.** Fresh clone builds & runs? Final commit, push, copy the link.
-

Setup checklist (quiet points)

- [] **.NET 8 (LTS) + EF Core 8** — they said ignore the .NET 5 note, use 6+.

- [] **SQLite** as the database — real relational DB, file-based, zero setup, so the reviewer clones and runs instantly. Say in the README you chose it for exactly that reason.
 - [] Add a .gitignore (dotnet new gitignore) so bin/, obj/ don't get committed.
 - [] Commit migrations to the repo; seed a little data so it runs out of the box.
 - [] Add an .editorconfig for consistent formatting.
 - [] First commit is an empty, building skeleton — then build up.
-

README template (paste this and fill it in)

```
# <Project Name>

## Overview
One paragraph: what the app does and the problem it solves.

## How to run
1. dotnet restore
2. dotnet ef database update --project <Infra project>    (or: it auto-migrates on startup)
3. dotnet run --project <Api project>
4. Open https://localhost:xxxx/swagger

## Tech choices
- .NET 8, EF Core 8, SQLite (file-based, zero-setup for review)
- <Web API / minimal API / console>, xUnit for tests
- Why: <one line each>

## Domain model
- Entities & relationships (a quick list or ASCII sketch)
- Aggregates / value objects used, and WHY

## Assumptions
- <every decision you made where the spec was unclear>

## Trade-offs & what I'd do with more time
- <what you deliberately left out and why>
- <what you'd harden for production: auth, validation, concurrency, pagination...>

## Testing
- What you tested and why those tests matter most
```

Commit message convention (Conventional Commits)

Format: type: short imperative summary. Types: **feat** (feature), **fix**, **refactor**, **test**, **docs**, **chore** (setup/tooling).

A sample commit story that mirrors the time plan — aim for something like this:

```
chore: initialise solution and project structure
chore: add gitignore and editorconfig
feat: add domain entities and relationships
feat: configure EF Core DbContext and Fluent mappings
chore: add initial migration and seed data
feat: implement <core operation> with <key business rule>
feat: expose API endpoints for <operations>
test: add unit tests for <business rule>
refactor: move <rule> into a domain method
docs: add README with setup, decisions and assumptions
```

Small and frequent beats one giant "final commit". Each message should say *what changed and why*, not "wip" or "stuff".

EF Core cheat-sheet

Scaffolding the solution (CLI):

```
dotnet new sln -n <Name>
dotnet new webapi -n <Name>.Api
dotnet new classlib -n <Name>.Domain
dotnet new classlib -n <Name>.Infrastructure
dotnet new xunit -n <Name>.Tests
dotnet sln add **/*.csproj
```

EF Core tooling & migrations (CLI):

```
dotnet tool install --global dotnet-ef
dotnet add <Infra> package Microsoft.EntityFrameworkCore.Sqlite
dotnet add <Infra> package Microsoft.EntityFrameworkCore.Design
dotnet ef migrations add InitialCreate --project <Infra> --startup-project <Api>
dotnet ef database update --project <Infra> --startup-project <Api>
dotnet ef migrations remove
```

Package Manager Console equivalents (Visual Studio):

```
Add-Migration InitialCreate
Update-Database
Remove-Migration
```

(Set the Infrastructure project as the "Default project" in the PMC dropdown, and the Api as the startup project.)

Query reflexes: - Reads you won't save → `.AsNoTracking()` - Avoid N+1 → `.Include()` or project with `.Select(...)` into a DTO - Multiple includes → `.AsSplitQuery()`

Likely review questions (the 45-minute follow-up)

Be ready to narrate, ending each on a trade-off: - Walk me through your domain model — why these entities, aggregates and value objects? - How did you implement the core business rule? Where does that logic live, and why there? - Why SQLite? What would you use in production, and what would change? - How would this scale? What breaks at 10× load and what would you do? - How did you handle concurrency / conflicting

updates? (e.g. two bookings for the same slot) - Did you use CQRS / DDD? Why or why not here? - How did you test it, and why those tests over others? - What did you deliberately leave out, and what would you add with more time? - If you had to put this live for a regulated, sensitive-data product, what would you add? (auth, audit, validation, encryption)

The candidate who models cleanly, documents assumptions, commits in a readable story, and explains every choice beats the one with more features and no narrative. That's the whole game.